# ✍️ Prompt & Golden Answer Writing

# Prompt & Golden Answer Writing

The prompt and golden answer should be designed together. The prompt should make the agent explore the repo; the golden answer should prove the expert path through the relevant evidence.

## Prompt Statement Guidelines

A prompt is the natural user-facing question the model will answer. It should give enough context for fair repo investigation while keeping the rubric and answer path out of view. Follow the guidelines below when writing your prompt.

| Guideline | Description | Examples |
|---|---|---|
| **Ask one clear engineering question** | The prompt should have a concrete target behavior, symptom, flow, or change to analyze. | ✅ Good:<br>“Why do mini-cart checkout actions reload the whole page while checkout-step transitions stay in-app?”<br><br>❌ Bad:<br>“Explain this repository.” |
| **Name useful starting points** | Mention files, modules, actions, patches, or symptoms when needed to make the task fair. | ✅ Good:<br>“Look at the mini-cart actions and where routes are declared.”<br><br>✅ Good:<br>“Across `composition.py`, `transforms_interface.py`, bbox/keypoint utils, and label manager…” |

| | | |
|---|---|---|
| **Do not paste the answer path** | Keep your discovered causal chain and downstream conclusion out of the prompt. | ✅ Good:<br>“What downstream behavior could this parser change affect?”<br><br>❌ Bad:<br>“This parser feeds the analytics view and breaks SQL filters. Explain why.” |
| **Avoid large copied code blocks** | The repo is provided as context. The model should inspect it directly. | ✅ Good:<br>Reference `pr.patch` as an artifact.<br><br>❌ Bad:<br>Paste the full patch into the prompt. |
| **Match the assigned intent** | The framing should match RCA, onboarding, PR triage, or architecture. | ✅ Root cause:<br>“What causes this observed reload?”<br><br>✅ PR review:<br>“What should I flag before approval?”<br><br>❌ Architecture prompt that only asks for a bug fix. |
| **Use realistic developer language** | The prompt should sound like a human debugging, reviewing, or onboarding to a real codebase. | ✅ Good:<br>“I’m new on this project. I’m trying to understand the normal execution flow…”<br><br>❌ Bad:<br>“As a world-class architect, produce a comprehensive treatise.” |

| | | |
|---|---|---|
| **Preserve answerability** | Include the symptom, patch, flow, or area of uncertainty needed to make the question solvable. | ✅ Good:<br>“What we see: full document navigation from those two actions. What we expected: SPA-style transition.”<br><br>❌ Bad:<br>“Something is wrong with checkout. Find it.” |

## Golden Answer Guidelines

The golden answer is the reference answer reviewers and rubrics are anchored to. It should be specific enough to support rubric criteria and polished enough to read as a clear expert response.

| Guideline | Description | Example / Tip |
|---|---|---|
| **Start with the answer** | Give the direct conclusion before the supporting details. | Example:<br>“The reload happens because the mini-cart handlers assign `window.location.href`, causing browser document navigation.” |
| **Show code evidence** | Name the files/functions/modules that prove the answer. | Examples:<br>• `miniCart.js` handlers.<br>• `Compose.__init__` processor creation.<br>• `migrate-source-files.ts` transform-array behavior. |
| **Explain the causal chain** | Connect evidence to outcome. | Explain more than “uses href”: React Router never sees the navigation intent, so the browser loads a new document. |

| | | |
|---|---|---|
| **Cover relevant alternatives** | Mention plausible wrong explanations when they clarify the answer. | Example:<br>GraphQL/useMiniCart explains cart contents only; the navigation mechanism comes from the click handlers. |
| **Support the rubric** | Write answer elements that can become atomic criteria. | Separate mechanism, functions, contrast path, affected files, and implications so the rubric can grade them separately. |
| **Avoid LLM-like filler** | Use concise, repo-grounded expert prose. | Cut out broad statements lacking repo evidence. |

## Prompt Examples

Good examples should force repo exploration without overfeeding the answer. Bad examples are either too broad, too leading, or too detached from the actual repository.

| Intent | Example | Commentary |
|---|---|---|
| ✅ **Root Cause Analysis** | Mini-cart reload example:<br>“What we see: full document navigation from Edit shopping bag / Proceed to checkout. What we expected: SPA routing. Trace which handlers implement those clicks and how their URL change differs from in-app navigation.” | Specific observed symptom.<br>Requires tracing handlers and routes.<br>Does not reveal `window.location.href`. |
| ❌ **Root Cause Analysis** | “The mini cart uses `window.location.href`; explain why that reloads the browser.” | Gives away the core mechanism.<br>Too close to the golden answer. |
| ✅ **Code Onboarding** | `Compose` flow example: | Specific multi-module flow. |

| | | |
|---|---|---|
| | “Walk through how `Compose` with bbox/keypoint params, label fields, aliases, and strict mode validates input, prepares processors, applies child transforms, filters, and postprocesses outputs.” | Still asks for explanation while keeping the answer path hidden. |
| ❌ **Code Onboarding** | “What does `Compose.__call__` do?” | Too local and likely answerable from one method. |
| ✅ **PR review** | Migration patch example:<br>“What is `pr.patch` doing under the covers, which parts of the codebase ripple from it, and what should I flag before approval?” | Natural reviewer question.<br>Forces blast-radius analysis. |
| ✅ **Architecture** | Architecture example:<br>“I want time-of-day autoscaling. Where should this fit architecturally, and how should config be defined?” | Asks for design placement and config mechanism only. |

# 📏 Rubric Writing & Difficulty Calibration

# Rubric Writing & Difficulty Calibration

A rubric is the grading contract for a task. It tells us exactly which facts, reasoning steps, and harmful failure modes a model response must include or avoid in order to count as correct.

For Agentic Code QA, a strong rubric translates the golden answer into independent pass/fail checks grounded in repository evidence.

## Target Difficulty Bands

- **Too Easy:** pass rate **85-100%** (not valid for this project)
- **Easy**: target pass rate **60–85%**
- **Medium**: target pass rate **35–60%**
- **Hard**: target pass rate **0–35%**
- All target ranges are **+/- 3%**

## Rubric Criteria

The independent parts of a rubric are called “criteria”. Each criterion is a true/false statement that defines a good response to the prompt (...or a bad one, in the case of penalty criteria).

### Criterion Writing Rules

Start by making each criterion precise enough to grade consistently. A reviewer should be able to read one criterion and decide whether the response earns it without guessing what you meant.

| Rule | Description | Examples |
|---|---|---|
| **Make each criterion atomic** | One criterion should test one observable behavior. Split distinct facts, mechanisms, and recommendations into separate criteria. | ✅ Good:<br>“States that the Edit shopping bag click triggers a `window.location.href` assignment in `miniCart.js`.”<br><br>❌ Bad:<br>“Explains checkout navigation and |

| | | |
|---|---|---|
| | | routing and reload behavior.” |
| **Use measurable wording** | A reviewer should know whether the response earns the point from the text alone. Avoid words like good, correct, appropriate, robust, and clear unless you anchor them to observable content. | ✅ Good:<br>“Explains that `_set_keys` expands the compose key registry before call-time validation.”<br><br>❌ Bad:<br>“Correctly explains validation.” |
| **Use concrete action verbs** | Start each criterion with a concrete, present-tense action verb so the grader knows what output to look for. | Examples:<br>States, Identifies, Explains, Includes, Distinguishes, Penalizes. |
| **Tie criteria to repo evidence** | Criteria should require evidence from the assigned codebase, PR, function, module, or behavior. | ✅ Good:<br>“Identifies that `migrateSourceFiles` now accepts a single transform or transform array.”<br><br>❌ Bad:<br>“Mentions relevant migration files.” |
| **Allow valid alternatives** | Allow alternative phrasing or file sequences when another correct path proves the same answer. | ✅ Tip:<br>Credit either function names or equivalent click-action names if the response connects both actions to browser navigation. |
| **Keep style non-critical** | Style can improve readability, but factual correctness should carry the task. | ✅ Good:<br>“Separates conclusion from evidence and implications.” (weight: bonus)<br><br>❌ Bad:<br>“Uses bullet points.” (weight: critical) |
| **Avoid placeholders or** | Every field must be final before submission. Placeholder text makes | Reject:<br>TODO, TBD, [description], empty |

| | | |
|---|---|---|
| **template residue** | the rubric ungradable. | rationale, copied template labels. |
| **Keep the golden answer aligned** | The golden answer should be able to earn every positive criterion and avoid every penalty. If the rubric changes, re-check the golden answer. | ✅ Good:<br>After adding a penalty for claiming a view was modified, confirm the golden answer correctly says the diff only introduces a function. |

### Criterion Examples

Use examples like these to make criteria atomic, measurable, and tied to the codebase.

Good criteria name the specific mechanism, behavior, or claim being evaluated.

| Type | Example | Why It Is Good / Bad |
|---|---|---|
| ✅ **Reasoning** | Root cause example:<br>“States that assigning a URL string to `window.location.href` causes the browser to perform a full document navigation.” | Tests the mechanism that produces the reload.<br>Grounded in the actual root cause example. |
| ✅ **Reasoning** | Code flow example:<br>“Explains that `Compose.add_targets` propagates alias mappings into child transforms.” | Specific processor/alias behavior.<br>Specific to Compose behavior. |
| ✅ **Completeness** | PR impact example:<br>“Identifies that `updateSourceFile` now applies an array of transforms sequentially and preserves the modified source between transforms.” | Concrete PR behavior.<br>Useful for blast-radius assessment. |
| ✅ **Penalty** | SQL rubric example:<br>Description: “Claims the view was modified in this PR.”<br>Rationale: The diff only introduces a new function, it doesn’t modify the view | Specific false claim.<br>Prevents inflated blast-radius reasoning. |

| | | |
|---|---|---|
| ✅ **Bonus** | “Formats the explanation using structured hierarchical elements like Markdown headers or numbered lists.” | Rewards proper formatting while not prescribing a single option. |
| ❌ **Vague criterion** | “Explains the issue correctly.” | Unmeasurable.<br>Fails to identify what the response must say. |
| ❌ **Compound criterion** | “Names the handler, explains routing, mentions GraphQL, and proposes a fix.” | Too many independent checks in one criterion.<br>Split into separate criteria. |

## Rubric Metadata

In addition to the rubric criterion, there are several other metadata fields that must be filled out to provide more information about your rubric. Each metadata field is described below.

### 1. Description

Fill `description` with the text of the pass/fail criterion. Follow all criterion guidelines above.

### 2. Rationale

Use `rationale` to explain why the criterion matters for grading this task. A good rationale connects the criterion to correctness, answerability, blast radius, or a common failure mode. It is for a human to read, and is not explicitly part of the test itself. **It is also relatively brief!** (We do not want an essay typed in the box.)

**Example rationale:** This criterion matters because the root-cause answer must distinguish the mini-cart click handler from GraphQL cart loading. Without that

distinction, the model can name nearby cart files while missing the reload mechanism being graded.

### 3. Tag

Use `tag` to identify the kind of criterion being evaluated. Choose the tag based on what the criterion tests, not where it appears in the answer.

| Tag | Use For | Example Criterion |
|---|---|---|
| **reasoning** | Cause, mechanism, tradeoff, code behavior, data flow, control flow, or architectural implication. | Identifies that repeated `VOLATILE` calls make the generated SQL nondeterministic. |
| **completeness** | Required factual answer elements that must appear for the answer to be complete. | Includes both mini-cart click actions and the file where the navigation behavior is defined. |
| **style** | Organization, clarity, or readability. Style should be a small minority of the rubric. | Provides headers separating conclusion, evidence, and implication. |

### 4. Weight

Use `weight` to set how the criterion affects grading. Most task-specific reasoning and required completeness criteria should be critical; optional nuance should be bonus; harmful false claims should be penalties.

| Weight | Use For | Tips |
|---|---|---|
| **critical** | Core correctness. Missing this point would make the answer materially incomplete or wrong. | Use for most task-specific reasoning and required completeness criteria. |
| **bonus** | Useful optional nuance that | Use for safe follow-up analysis, |

| | | |
|---|---|---|
| | improves the answer but is not required for correctness. | nonessential edge cases, or an extra implication. Also commonly used for style criteria. |
| **penalty** | Harmful false claims, misleading shortcuts, or bad engineering recommendations. | Penalizes claiming a PR changed a UI view when the diff only adds a helper function. |

### 5. Dependencies

Use `dependent_on` only when a criterion cannot be evaluated fairly unless another criterion is met first. For example, a downstream blast-radius criterion may depend on first identifying the correct modified function. Leave this field empty when the criterion can be graded on its own. Never make a critical criterion dependent on a bonus criterion.

## Rubric Criteria Limits

| Criterion Type | Required Range |
|---|---|
| Critical | 7-23 (Mandatory to have both reasoning and completeness criteria as critical) |
| Bonus | 4-17 |
| Style | 2-6 |
| Penalty | 5-10 |
| *Total Criteria* | ~35 target (40 maximum for harder tasks) |

## Using Qualifiers in Rubric Criteria

When a single requirement covers multiple conditions within the same case, use explicit qualifiers like “all,” “every,” or “each.” In some cases, you may use qualifiers like “one or more of” and then a list of comma-delimited options.

Qualifiers make it clear when the requirement is met if only a subset of conditions is satisfied.

❌ **Ambiguous:** The API exposes GET, POST, and PATCH methods

✅ **Clear:** The API exposes all three methods: GET, POST, PATCH

In the ambiguous example, it is a bit unclear whether the requirement is “one or more from the list” or “all of these.” Be explicit. Rubric evaluations tend to flip (change score) between runs when qualifiers are not clearly stated, leading to inaccurate grading.

## QC & Calibration

### Airtable QC & Submission Checks

We have built in a series of LLM tools within Airtable to help you check over your submission before sending it to a reviewer. Use these LLM checks as supporting signals before submission.

Human judgment remains required: read the outputs and revise only when the finding is valid.

| Check | When To Run | How To Use The Result |
|---|---|---|
| **Prompt QC** | After drafting prompt. | Read the output, then use human judgment. Keep prompt wording under your control and avoid answer hints. |
| **Rubric QC** | After drafting rubric. | Look for non-atomic criteria, vague wording, missing penalties, or rubric/golden-answer mismatch. Revise the rubric and golden answer together when needed. |

| | | |
|---|---|---|
| **Golden Answer QC** | When both the golden answer and rubric are populated. | Check whether the answer fully resolves the prompt from repo evidence. Treat the QC result as a signal that still requires human judgment. |
| **Generate and Evaluate Model / New Test Model** | After validation steps above pass. | This is the source of truth for difficulty. It may take 5-10 minutes or longer. Use the true score evaluation at the bottom. The run averages 3 evaluation runs but only prints detailed results from the first run. |
| **Submit Your Task** | After manual review, QC checks, validation, and difficulty calibration are complete. | Submit in Airtable only when the task is ready for reviewer inspection. If this is your first task, wait for reviewer approval before claiming another. Address revisions before starting new work. |

### Difficulty Calibration

Each task is associated with a given difficulty level, and you will have to calibrate your task to ensure that the finished task lands at the intended level of model challenge.

You will run a model eval that scores the model answer against the weighted rubric. To pass difficulty calibration, your rubric pass rate must align with the target difficulty.

| Difficulty | Target Pass Rate | Typical Task Shape |
|---|---|---|
| **Too Easy** | >85% | Simple lookups or surface questions. Tasks in the Too Easy scoring band are not valid for this project. |
| **Easy** | 60-85% (+/- 3%) | Localized repo question; usually answerable from 1-2 files without running code. |
| **Medium** | 35-60% (+/- 3%) | Cross-file reasoning, data/control flow, abstraction behavior, or moderate PR impact analysis. |
| **Hard** | 0-35% (+/- 3%) | Multi-hop reasoning, subtle edge cases, architecture tradeoffs, executable evidence, or system-level |

| | | |
|---|---|---|
| | | impact. |

### Evaluation Signal

Use both the scored answer and the model trajectory when judging difficulty. A task can have the right pass rate for the wrong reason if the prompt is ambiguous, the rubric is misaligned, or the model succeeds without meaningful repo exploration.

| Signal | What To Check | How To Interpret |
|---|---|---|
| **Step 5 score** | Use the Step 5 eval score before submission. This score includes rubric weights and penalties. | The score must land in the assigned band, with +/- 3% tolerance. Scores above 85% mean the task is too easy. |
| **Golden validation** | Confirm Validate Golden passes and the golden answer scores 100%. | A low golden score usually means the rubric and golden answer are misaligned. Fix that, run again. (See Golden Evaluation below.) |
| **Prompt fairness** | Check whether the prompt gives enough starting context without revealing the answer path. | Missing context makes the task unfair. Answer breadcrumbs make the task too easy. |
| **Intent fit** | Compare the final task to the assigned intent and difficulty target. | If the content no longer matches the assigned intent, retarget the prompt or flag the task rather than forcing calibration. |

### Calibration Fixes

When calibration misses the target, revise the prompt, golden answer, or rubric based on the reason it missed. Do not force a task into the assigned band by hiding necessary context or accepting generic answers.

| Result | Likely Cause | Fixes |
|---|---|---|
| **Too easy** | Prompt gives away files/conclusion, answer is a local lookup, rubric accepts generic answers, or Step 5 passes with minimal exploration. | Remove answer breadcrumbs.<br>Require deeper repo evidence.<br>Scope toward a non-obvious interaction, risk, or causal chain. |
| **Too hard or ambiguous** | Prompt lacks needed symptom/file/patch context, rubric demands unsupported detail, or the answer path depends on hidden assumptions. | Clarify starting context.<br>Add necessary references.<br>Relax over-specific criteria that are not required for correctness. |
| **Wrong intent** | Task content no longer matches the assigned intent. | Retarget the prompt to the assigned intent or flag the task if the assignment itself is wrong. |
| **Rubric mismatch** | Golden answer does not achieve 100%, valid alternatives are penalized, or penalties distort the Step 5 score. | Rewrite criteria from the final golden answer.<br>Add fair alternative paths.<br>Check penalty weights before recalibrating. |

### Golden Evaluation

Golden answers were graded against the same rubric to verify rubric quality. Expected outcome: all critical criteria pass, all bonuses achievable, no penalties.

| Metric | Value | Definition |
|---|---|---|
| Pass Rate | 100% (100/100) | % of tasks where ALL critical criteria pass AND zero penalties triggered |
| Excellence Rate | 100% (100/100) | % of tasks where ALL critical pass AND ≥1 bonus passes AND zero penalties |

| | | |
|---|---|---|
| Violation Rate | 0% (0/100) | % of tasks with ≥1 penalty triggered (penalty “passes” = bad behavior present) |

### Tips for Making Hard Tasks

Triggering model failure is one of the hardest parts of this project. Follow the tips below to drive up task difficulty:

- **Start from research before writing the prompt.** Spend the first part of task creation understanding the repo, the assigned intent, relevant files, tests, issues, docs, and common user pain points. Hard tasks usually come from knowing enough context to ask about the non-obvious interaction a model would not naturally prioritize. Devote significant resources to research so that you have the knowledge to ask a hard question.
- **Give fair starting context while preserving discovery.** A long prompt that names every file, function, lifecycle step, and suspected cause can make a hard repo question easy. Provide the symptom, goal, patch, or subsystem the developer would realistically know, then let the model discover the exact evidence chain.
- **Trace indirect effects and downstream dependencies.** For PR triage, move beyond summarizing the diff: apply the patch mentally, trace where changed functions are called, and ask about the maintainability, correctness, security, or compatibility risk that appears after following those dependencies.
- **Make architecture tasks about real constraints.** Strong architecture prompts come from requested features, recurring user needs, or current repo limitations. Ask how the design should fit the existing modules, configuration, interfaces, and deployment constraints, so the answer requires tradeoff reasoning instead of generic design advice.
- **Make onboarding tasks explain a concrete flow or implementation path.** Generic comprehension prompts are easy because the model can summarize nearby code. Harder onboarding asks the model to explain how several pieces

work together, where a change would enter the system, or why a behavior emerges from aliases, processors, config, routes, state, or validation.

- **Make root-cause tasks distinguish plausible wrong explanations.** A hard root-cause prompt gives a realistic failing behavior and enough context to investigate, then requires the answer to trace the exact mechanism and rule out tempting but incorrect causes. Keep the file, line, and bug source out of the prompt.
- **Recalibrate after hardening.** If the task becomes hard because context is missing, add the missing context. If it remains easy because the model succeeds with shallow exploration, remove answer breadcrumbs or scope toward a deeper interaction. If it becomes unfair, repair the prompt before rerunning Step 5.